Clase asincrónica N°25 – DB2

OBJETIVO: Al recorrer este documento el estudiante podrá comprender los conceptos básicos de la base de datos relacional DB2.

¿Qué es una base de datos DB2?

- DB2 es la base de datos relacional desarrollada por IBM, ampliamente utilizada en el entorno de mainframe para gestionar grandes volúmenes de datos.
- Una base de datos DB2 es un conjunto de datos tratados colectivamente como una unidad.
- Una base de datos DB2 tiene el propósito de almacenar, recuperar y manipular datos relacionados entre sí.

¿Para qué sirve una base de datos DB2?

- Utilizada por organizaciones de todos los tamaños, DB2 proporciona una plataforma de datos para operaciones tanto transaccionales como analíticas, así como disponibilidad continua de datos para mantener los flujos de trabajo transaccionales y sus respectivos análisis operando de manera eficiente.

Características de DB2

Una base de datos DB2 tiene las siguientes características:

- *Motor SQL común*: Una consulta puede escribirse una vez y usarse en todos los productos y plataformas.
- *Puede admitir todos los tipos de datos*: Se puede acceder a los datos estructurados, no estructurados y relacionales en una plataforma.
- *Disponibilidad alta y recuperación ante desastres*: La funcionalidad de replicación de Db2 permite un almacenamiento y acceso seguro.
- *Escalabilidad*: Los usuarios pueden ampliar el almacenamiento local, y también escalar el almacenamiento y la energía en una nube administrada para ahorrar dinero.
- *Partición de tablas*: En un almacén de datos (*data warehouse*) de Db2, la característica de partición de bases de datos permite a los usuarios dividir datos entre servidores para maximizar la potencia informática y permitir el procesamiento paralelo.

Ejemplos de DB2

- Disponibilidad y escalabilidad para grandes empresas
- Almacenamiento de datos empresariales que constituyen el origen de aplicaciones críticas
- Distribución de los datos y acceso WEB
- Garantiza servicios ininterrumpidos a sus clientes durante las 24 hs del día
- Un programa puede acceder a la base de datos para recuperar información

¿Qué es normalización de base de datos?

La Normalización de Base de Datos es un principio de diseño de Base de Datos para organizar los datos de una manera consistente y estructurada.
Ayuda a evitar redundancia y mantener la integridad de la Base de Datos. También ayuda a eliminar características indeseables asociadas con la inserción, eliminación, y actualización.

¿Cuál es el propósito de la normalización?

El propósito principal de la Normalización de Base de Datos es evitar complejidades, eliminar duplicados, y organizar los datos de una manera consistente. En Normalización, los datos están divididos en varias tablas enlazadas juntas con relaciones.

Los administradores de Base de Datos son capaces de lograr estas relaciones usando las claves primarias y foráneas o alternativas.

Para hacerlo, una clave primaria en una tabla, por ejemplo, CLIENTES está relacionada con el valor de la otra tabla, por ejemplo: PERSONAS.

Nota: **Una clave primaria** es una columna/s que identifican únicamente las filas de datos en esa tabla. Es un identificador único tal como ID de cliente, Tipo y nro. documento, y así sucesivamente.
Una **clave foránea** es un campo o columna que se relaciona a la clave primaria de la otra tabla.

Ejemplo:
Clave primaria en CLIENTES: ID del cliente

Su clave foránea es Tipo y nro. de documento que se refiere a la clave primaria de PERSONAS

¿Qué son las formas normales o normalización de bases de datos?

Primera, segunda o tercera forma normal; son los tres primeros tipos de normalización de base de datos.

Hay también cuarta forma normal y quinta forma normal. Hay inclusive sexta forma normal, pero la forma normal más común que verás será tercera forma normal.

Todos los tipos de normalización de base de datos son acumulativos – lo que quiere decir es cada uno se construye uno sobre el otro. Así que todos los conceptos en primera forma normal también llevan a la segunda forma normal, y así sucesivamente.

La primera forma normal – 1NF
Para una tabla ser la primera forma normal, debe cumplir el siguiente criterio:

- una sola celda no debe contener más de un valor
- debe haber una clave primaria para identificación
- no filas o columnas duplicadas
- cada columna debe tener solamente un valor por cada fila en la tabla

La segunda forma normal – 2NF
El 1NF solamente elimina los grupos repetitivos, no la redundancia. Por eso hay 2NF.
Una tabla se dice que está en 2NF si cumple el siguiente criterio:

- ya está en 1NF
- no tiene dependencia parcial. Es decir, todos los atributos no claves son totalmente dependientes de la clave primaria

La tercera forma normal – 3NF
Cuando una tabla está en 2NF, elimina los grupos repetitivos y la redundancia, pero no elimina la dependencia parcial transitiva.
Esto significa que un atributo no principal (un atributo que no forma parte de la clave del candidato) es dependiente de otro atributo no principal. Esto es lo que la tercera forma normal (3NF) elimina.
Así que, para que una tabla está en 3NF, debe:

- estar en 2NF
- no tiene dependencia parcial transitiva

Ejemplo

Base de datos CLIENTES:

Sus columnas o atributos son:

```
( TIPDOC        CHAR(2) NOT NULL,
  NRODOC        DECIMAL(11, 0) NOT NULL,
  NROCLI        DECIMAL(3, 0) NOT NULL,
  NOMAPE        CHAR(30) NOT NULL,
  FECNAC        DATE NOT NULL,
  SEXO          CHAR(1) NOT NULL )
```

CHAR(2): indica que el tipo de datos es carácter de 2 bytes; NOT NULL. Es porque este atributo no acepta caracteres nulos
DECIMAL(11 0): indica que es un campo PIC 9(11).
DATE: indica que es un fecha AAAA-MM-DD

EL ALCANCE DE ESTA CAPACITACIÓN SE REFIERE A LA UTILIZACIÓN DE BASES DE DATOS YA DEFINIDAS; LAS CUALES SERÁN UTILIZADAS POR LOS ESTUDIANTES PARA EL DESARROLLO DE ACTIVIDADES.

PROBANDO LAS COMPILACIONES y AUTORIZACIONES POR CADA ESTUDIANTE

En este capítulo estaremos comenzando en forma coloquial e intuitiva a compilar un programa COBOL con acceso a los datos a través de bases de datos de CLIENTES y de CUENTAS.

A lo largo de esta capacitación estaremos trabajando con estas bases de datos que ya están pobladas con datos:
1 - **CLIENTES** KC02803.TBCURCLI donde el primer calificador del nombre es el OWNER o propietario de dicha tabla de datos que nos autoriza a accederla

2 - **CUENTAS** KC02803.TBCURCTA donde el primer calificador del nombre es el OWNER o propietario de dicha tabla de datos que nos autoriza a accederla

Para conocer la estructura de cada una de ellas deberán copiar:

1 - KC02788.ALU9999.DCLGEN(TBCURCLI)

2- KC02788.ALU9999.DCLGEN(TBCURCTA)

Para ello, previamente deberán alocar la biblioteca personal **KC03XXX.CURSOS.DCLGEN** copiando la estructura de KC02788.ALU9999.DCLGEN

🧠 ¿Qué es DCLGEN?

DCLGEN significa *Declaration Generator*. Es una herramienta que genera automáticamente:

- **Sentencias SQL DECLARE TABLE**: para definir la estructura de una tabla en SQL.
- **Estructuras de datos en COBOL o PL/I**: que reflejan esa misma tabla, listas para ser usadas en tu código.

🎯 ¿Para qué se utiliza?

Principalmente para **facilitar la integración entre DB2 y programas con código COBOL.**

Sus usos más importantes son:

- ✅ **Evitar errores de tipeo**: al generar automáticamente las estructuras, se reduce el riesgo de inconsistencias entre la base de datos y el código.
- ✅ **Acelerar el desarrollo**: no tenés que escribir manualmente cada campo de la tabla en tu programa.

- ✅ **Sincronizar estructuras**: asegura que los nombres, tipos de datos y longitudes coincidan entre DB2 y tu código fuente.
- ✅ **Facilitar el uso de SQL embebido**: como en sentencias `SELECT INTO`, donde necesitás variables que coincidan con los campos de la tabla.

📌 Ejemplo práctico

Supongamos que tenés una tabla `CLIENTES` en DB2. Usando DCLGEN, podrías generar:

```
UBA - wc3270
File    Options    Keypad

File  Edit  Edit_Settings  Menu  Utilities  Compilers  Test  Help
EDIT       KC02788.ALU9999.DCLGEN(TBCURCLI) - 01.01        Columns 00001 00072
Command ===> _____________________________________________ Scroll ===> CSR_
****** ***************************** Top of Data ******************************
000001        ********************************************************************
000002        * DCLGEN TABLE(KC02803.TBCURCLI)                                  *
000003        *        LIBRARY(KC02788.CURSOS.DCLGEN(TBCURCLI))                 *
000004        *        ACTION(REPLACE)                                          *
000005        *        LANGUAGE(COBOL)                                          *
000006        *        QUOTE                                                    *
000007        * ... IS THE DCLGEN COMMAND THAT MADE THE FOLLOWING STATEMENTS    *
000008        ********************************************************************
000009             EXEC SQL DECLARE KC02803.TBCURCLI TABLE
000010             ( TIPDOC                         CHAR(2) NOT NULL,
000011               NRODOC                         DECIMAL(11, 0) NOT NULL,
000012               NROCLI                         DECIMAL(3, 0) NOT NULL,
000013               NOMAPE                         CHAR(30) NOT NULL,
000014               FECNAC                         DATE NOT NULL,
000015               SEXO                           CHAR(1) NOT NULL
000016             ) END-EXEC.
000017        ********************************************************************
000018        * COBOL DECLARATION FOR TABLE KC02803.TBCURCLI                     *
000019        ********************************************************************
000020         01  DCLTBCURCLI.
000021             10 WT-TIPDOC                PIC X(2).
000022             10 WT-NRODOC                PIC S9(11)V USAGE COMP-3.
000023             10 WT-NROCLI                PIC S9(3)V USAGE COMP-3.
000024             10 WT-NOMAPE                PIC X(30).
000025             10 WT-FECNAC                PIC X(10).
000026             10 WT-SEXO                  PIC X(1).
000027        ********************************************************************
000028        * THE NUMBER OF COLUMNS DESCRIBED BY THIS DECLARATION IS 6        *
000029        ********************************************************************
****** ***************************** Bottom of Data ***************************

 F1=Help      F2=Split     F3=Exit      F4=Expand    F5=Rfind     F6=Rchange
 F7=Up        F8=Down      F9=Swap      F10=Left     F11=Right    F12=Cancel
B                                                    TS000006          005/015
```

Así, podés hacer un `SELECT INTO` directamente sobre esas variables dentro de un programa con código COBOL.

Este ejemplo lo podrás visualizar específicamente para las tablas destinadas a esta capacitación dentro de tu biblioteca personal DCLGEN.

Verás un ejemplo en:

KC02788.ALU9999.FUENTE(PGMDB210)

🛠️ Paso a paso: Uso de DCLGEN en DB2

1. 📋 Identificar la tabla

Antes de usar DCLGEN, tenés que saber el nombre de la tabla en DB2 con la que vas a trabajar. Ejemplo: `CLIENTES`.

---

2. 🧑‍💻 Acceder al entorno TSO/ISPF

En un entorno mainframe típico, accedés a **TSO/ISPF** y seleccionas la opción para ejecutar DCLGEN. Esto puede variar según la configuración, pero suele estar en el panel de utilidades.

---

3. 🧾 Completar los parámetros de DCLGEN

En el panel de DCLGEN, vas a ingresar:

- **Nombre de la tabla**: `CLIENTES`
- **Nombre de la DCLGEN dentro de host**: por ejemplo, `DCLCLIENTES`
- **Biblioteca de salida para el código COBOL**: donde se guarda el COPY
- **Biblioteca de salida para el SQL**: donde se guarda el DECLARE TABLE

---

4. ⚙️ Ejecutar DCLGEN

Una vez completados los campos, se podrá ejecutar el proceso. Esto genera dos archivos:

- Un **COPY** COBOL con la estructura de la tabla (`DCLCLIENTES`)
- Un **DECLARE TABLE** SQL que podés incluir en tu programa

---

5. 📦 Incluir los archivos en tu programa COBOL

En tu código COBOL, se utilizará:

EXEC SQL INCLUDE DCLCLIENTES END-EXEC.

Y también se incluirá el `INCLUDE SQLCA` para poder utilizar SQL embebido.

---

6. 🧪 Validar tipos de datos y nombres

Es buena práctica revisar que los tipos de datos generados coincidan con lo que esperás. Por ejemplo:

```
01 DCLCLIENTES.
   05 ID-CLIENTE     PIC S9(9) COMP.
   05 NOMBRE         PIC X(50).
   05 CIUDAD         PIC X(30).
```

---

7. 🧵 Usar las variables en sentencias SQL

Ahora podés hacer cosas como:

```
EXEC SQL
   SELECT NOMBRE, CIUDAD
   INTO :NOMBRE, :CIUDAD
   FROM CLIENTES
   WHERE ID_CLIENTE = :ID-CLIENTE
END-EXEC.
```

🧭 Consejos prácticos

- 🔄 Si la tabla cambia, **SE DEBE REGENERAR el DCLGEN** para mantener la sincronización.
- 🧹 Mantener los COPYs organizados por tabla y programa.
- 🧪 Probar con sentencias simples antes de integrar lógica compleja.

🧵 Proceso completo para compilar COBOL con SQL embebido (DB2)

1. 🧾 Precompilación con el Precompiler de DB2

Este paso convierte las sentencias SQL embebidas en llamadas a rutinas DB2 y genera dos cosas:

- Un **programa COBOL limpio** con llamadas a DB2 (donde se han eliminado las sentencias SQL).
- Un **archivo DBRM (Database Request Module)** con las sentencias SQL.

Para ello, previamente deberán alocar la biblioteca personal **KC03XXX.CURSOS.DBRMLIB** copiando la estructura de KC02788.ALU9999.DBRMLIB

---

2. 🧪 Compilación COBOL

Con estas salidas del pre compilador; se ejecuta la compilación del código COBOL

Para ejecutar la precompilación y la compilación COBOL se podrá ejecutar el siguiente JCL; que copiarán a sus respectivas bibliotecas con los cambios y adecuaciones necesarias::

KC02788.ALU9999.FUENTE(COMPCDB2)

---

3. 📦 BIND (Enlace)

Este paso une el código compilado con las sentencias SQL que fueron descartadas dentro de la pre-compilación; armando el mejor camino de acceso desde el código COBOL a la base de datos DB2.

Este paso se podrá ejecutar desde el siguiente JCL; que copiarán a sus respectivas bibliotecas con los cambios y adecuaciones necesarias:

KC02788.ALU9999.FUENTE(BINDESTU)

```
TSO - wc3270
File      Options    Keypad
  File  Edit  Edit_Settings  Menu  Utilities  Compilers  Test  Help
VIEW       KC02788.ALU9999.FUENTE(COMPCDB2) - 01.11        Columns 00001 00072
****** ********************************* Top of Data *********************************
000001 //KC03XXXA JOB CLASS=A,MSGCLASS=O,MSGLEVEL=(1,1),NOTIFY=&SYSUID
000002 //*****************************************************
000003 //*   COMPILADOR COBOL/DB2 BATCH                      *
000004 //*   XX =    DIGITAR SU USERID ALUMNO                *
000005 //*   GOPGM DIGITAR SU PROGRAM-ID                     *
000006 //*****************************************************
000007 //JCLLIB JCLLIB ORDER=KC02788.ALU9999.PROCLIB
000008 //STEP1  EXEC COMPDB2,
000009 //       ALUMLIB=KC03XXX.CURSOS,
000010 //       GOPGM=PGMDB210
000011 //PC.SYSLIB  DD DSN=KC03XXX.CURSOS.DCLGEN,DISP=SHR
000012 //COB.SYSLIB DD DSN=KC02788.ALU9999.COPYLIB,DISP=SHR
****** ********************************* Bottom of Data ******************************
Command ===> _____________________________________________ Scroll ===> CSR_
 F1=Help      F2=Split     F3=Exit      F4=Expand    F5=Rfind     F6=Rchange
 F7=Up        F8=Down      F9=Swap     F10=Left     F11=Right    F12=Cancel
  *DSLIST
4B                                                TSO00008          040/015
```

4. 📚 Bind del DBRM

El archivo DBRM generado en el paso 2 se debe **bindear** para que DB2 lo reconozca como un paquete ejecutable.

Se podrá ver en:

KC02788.ALU9999.FUENTE(BINDESTU)

```
UBA - wc3270
ile      Options    Keypad

 File  Edit  Edit_Settings  Menu  Utilities  Compilers  Test  Help

EDIT       KC02788.ALU9999.FUENTE(BINDESTU) - 01.09        Columns 00001 00072
Command ===> _____________________________________________ Scroll ===> CSR_
****** ***************************** Top of Data ******************************
==MSG> -Warning- The UNDO command is not available until you change
==MSG>           your edit profile using the command RECOVERY ON.
000001 //KC03XXXD JOB CLASS=A,MSGCLASS=O,MSGLEVEL=(1,1),NOTIFY=&SYSUID
000002 //JOBLIB    DD  DSN=DSND10.SDSNLOAD,DISP=SHR
000003 //STEP1    EXEC PGM=IKJEFT01,DYNAMNBR=20,COND=(4,LT)
000004 //*******************************************************
000005 //*                                                     *
000006 //* DBRMLIB  DD  DSN=KC03XXX.CURSOS.DBRMLIB,DISP=SHR    *
000007 //*                                                     *
000008 //* BIND PLAN ES SIEMPRE    CURSOXXX   O J O !!!        *
000009 //* DONDE XXX = ULTIMAS LETRAS USER ID                  *
000010 //*                                                     *
000011 //* BIND MEMBER TIPEAR EL NOMBRE DEL PROGRAMA           *
000012 //*                                                     *
000013 //*******************************************************
000014 //DBRMLIB  DD  DSN=KC03XXX.CURSOS.DBRMLIB,DISP=SHR
000015 //SYSTSPRT DD SYSOUT=*
000016 //SYSTSIN  DD *
000017   DSN SYSTEM(DBDG)
000018   RUN  PROGRAM(DSNTIAD) PLAN(DSNTIA13) -
000019        LIB('DSND10.DBDG.RUNLIB.LOAD')
000020   BIND PLAN(CURSOXXX) MEMBER(PGMDB210) +
000021       CURRENTDATA(NO) ACT(REP) ISO(CS) ENCODING(EBCDIC)
000022   END
000023 //SYSPRINT DD SYSOUT=*
000024 //SYSUDUMP DD SYSOUT=*
000025 //SYSIN    DD *
****** ***************************** Bottom of Data ***************************

 F1=Help      F2=Split     F3=Exit      F4=Expand    F5=Rfind     F6=Rchange
 F7=Up        F8=Down      F9=Swap     F10=Left     F11=Right    F12=Cancel
3                                                   TS000006        004/015
```

🧠 ¿Por qué es importante cada paso?

Paso	Propósito
Precompilación	Traduce SQL embebido a llamadas DB2 y genera el DBRM
Compilación	Convierte el código COBOL en objeto ejecutable
Link-edit	Prepara el programa COBOL para ejecución batch KC02788.ALU9999.FUENTE(**COMPCDB2**)
Bind	Registra las sentencias SQL en DB2 para que puedan ejecutarse correctamente. KC02788.ALU9999.FUENTE(**BINDESTU**) Arma el camino lógico más óptimo entre el código COBOL y la tabla DB2 que se está accediendo.

Entonces vamos a probar nuestra autorización para acceso a las tablas de esta capacitación:

1- COPIAR programa cobol con DB2 de ejemplo a sus bibliotecas personales:

KC02788.ALU9999.FUENTE(**PGMDB210**)

2- Ejecutar la compilación del programa con el miembro copiado y adaptado a cada user id: **COMPCDB2**

3- Luego de compilado en forma exitosa. Ejecutar BIND del programa con el miembro copiado y adaptado a cada user id:

KC02788.ALU9999.FUENTE(**BINDESTU**)

Si todo resultó satisfactorio; entonces estamos en condiciones de seguir avanzando en el conocimiento que abarca esta capacitación.